Дисциплина: Тестирование программного обеспечения серверных платформ / Сетевые ОС

Тема: Разработка сценария нагрузочного тестирования с использованием Locust

Студент: dra1ner (Ильиных Дмитрий, ip414s07@csc.sibsutis.ru)

Окружение: Python 3.11, Locust 2.46.7, Requests, OpenBMC Romulus Redfish API, JSONPlaceholder, wttr.in

ЛАБОРАТОРНАЯ РАБОТА №6

Разработка сценария нагрузочного тестирования с использованием Locust

1. Цель и задачи работы

Цель работы: научиться разрабатывать сценарии нагрузочного тестирования с использованием фреймворка **Locust**, выполнив профилирование производительности REST API серверной платформы **OpenBMC** и открытых публичных веб-сервисов (**JSONPlaceholder**, **wttr.in**). Оценить пропускную способность системы (RPS), время отклика (Response Time), перцентили задержек и потенциальные точки отказа под воздействием одновременных запросов виртуальных пользователей.

Задачи:

1. Изучить теоретические основы нагрузочного тестирования и принципы работы фреймворка Locust (модель событийного асинхронного выполнения на корутинах `gevent`, классы `HttpUser`, распределение весов задач `@task`).
2. Подготовить тестовое окружение: установить пакет `locust` и зависимые модули.
3. Проверить доступность интерфейса Redfish API OpenBMC (с использованием Basic Auth и утилиты `curl`), а также внешних публичных сервисов (JSONPlaceholder, wttr.in).
4. Разработать скрипт нагрузочного тестирования `locustfile.py` с двумя классами нагрузки:
   - **Класс 1 (OpenBMC API):** создание сессии авторизации в хуке `on_start` (получение `X-Auth-Token`), периодический опрос паспорта системы (`/redfish/v1/Systems/system`), опрос состояния питания (`PowerState`) и тепловых датчиков (`/redfish/v1/Chassis/chassis/Thermal`);
   - **Класс 2 (Публичные API):** запрос списка публикаций на JSONPlaceholder (`/posts`) и запрос метеорологической сводки для Новосибирска на wttr.in (`https://wttr.in/Novosibirsk?format=j1`).
5. Выполнить нагрузочное тестирование в интерактивном режиме с веб-интерфейсом Locust (`http://localhost:8089`) и в автоматизированном консольном режиме (headless) с выгрузкой отчётов и CSV-метрик.
6. Провести детальный анализ метрик: среднее время отклика, медиана, 90%, 95% и 99% перцентили, пропускная способность, процент отказов. Определить максимальную стабильную нагрузку и точки отказа.
7. Оформить отчёт по лабораторной работе и приложить ссылку на репозиторий GitHub.

2. Теоретические сведения

Нагрузочное тестирование (Load Testing) — вид тестирования программного обеспечения, предназначенный для оценки поведения, устойчивости и времени отклика системы при воздействии ожидаемой или пиковой нагрузки, формируемой множеством параллельных пользователей или процессов.

Locust — современный инструмент нагрузочного тестирования с открытым исходным кодом, в котором сценарии поведения пользователей описываются чистым кодом на языке Python. В отличие от классических инструментов на базе потоков ОС (таких как Apache JMeter), Locust использует легковесные микропотоки (корутины) библиотеки `gevent`. Это позволяет одному экземпляру Locust без существенных накладных расходов генерировать тысячи конкурентных HTTP-соединений с одного узла.

Ключевые концепции Locust:

- `HttpUser` — базовый класс виртуального пользователя, предоставляющий клиента `client` (обёртка над `requests.Session`) для отправки HTTP-запросов и сбора телеметрии;
- `@task(weight)` — декоратор, объявляющий метод сценарием взаимодействия. Параметр веса задаёт относительную частоту выбора задачи;
- `wait_time = between(min, max)` — пауза между итерациями виртуального пользователя, имитирующая реальное поведение оператора (Think Time);
- `catch_response=True` — механизм контекстного менеджера, позволяющий анализировать содержимое ответа (JSON-схему, статус-код) и явно фиксировать успех или сбой (`response.success()` / `response.failure()`).

Специфика тестируемых сервисов:

- **OpenBMC Redfish API:** интерфейс управления сервером (драйвер `bmcweb`). Так как BMC функционирует на энергоэффективном процессоре архитектуры ARM с ограниченным объёмом оперативной памяти (256–512 МБ), частые запросы вызывают межпроцессные взаимодействия через системную шину `systemd / D-Bus`. Нагрузочное тестирование выявляет стабильность демонов телеметрии при непрерывном опросе.
- **JSONPlaceholder:** публичный mock REST API, оптимизированный для тестирования клиентских библиотек. Имеет кэширование на уровне CDN и стабильное время ответа.
- **wttr.in:** публичный консольный и веб-сервис погоды. Предоставляет сложные вычисления метеомоделей и накладывает строгие ограничения на частоту запросов (Rate Limiting), возвращая код `429 Too Many Requests` при агрессивной нагрузке.

3. Архитектура и структура разработанного проекта

```
openbmc_lab6/
├── locustfile.py             # Основной сценарий Locust (OpenBMCUser и PublicAPIUser)
├── openbmc_server.py         # Сервер-эмулятор Redfish API OpenBMC (DMTF v1.8)
├── run_load_tests.py         # Автоматизированный запуск headless-тестов со сбором статистики
├── results/                  # Каталог сгенерированных отчётов и сырых данных
│   ├── openbmc_report.html   # Интерактивный HTML-отчёт Locust по OpenBMC API
│   ├── public_report.html    # Интерактивный HTML-отчёт по публичным API
│   ├── combined_report.html  # Комплексный HTML-отчёт одновременной нагрузки
│   ├── combined_stats_stats.csv # Детальные числовые метрики всех запросов
│   └── combined_stats_failures.csv # Журнал отказов (0 сбоев)
├── README.md                 # Инструкция по установке, запуску и настройке
├── report_lab6.html          # Исходный HTML-документ отчёта
└── Отчет_Лабораторная_работа_6.pdf # Чистовой отчёт в формате PDF
```

4. Подготовка окружения и проверка доступности эндпоинтов

Установка зависимостей выполнена через пакетный менеджер `pip`:

```
python -m pip install locust requests
```

Предварительная проверка доступности корневого сервиса OpenBMC Redfish API:

```
curl -k -u root:0penBmc https://127.0.0.1:2443/redfish/v1/
```

Сервер возвращает статус `200 OK` и схему ServiceRoot. Аналогично выполнена проверка публичных эндпоинтов:

```
# Проверка JSONPlaceholder
curl -I https://jsonplaceholder.typicode.com/posts -> HTTP/2 200 OK

# Проверка wttr.in
curl -I https://wttr.in/Novosibirsk?format=j1 -> HTTP/2 200 OK
```

5. Реализация сценариев нагрузочного тестирования (`locustfile.py`)

5.1. Класс нагрузки `OpenBMCUser`

Класс моделирует обращение системных администраторов и агентов мониторинга к платформе OpenBMC. При старте виртуального пользователя в методе `on_start` отправляется POST-запрос авторизации для получения сессионного токена `X-Auth-Token`. Задачи распределены по весам: опрос информации о сервере (вес 3), проверка состояния питания (вес 2) и чтение термодатчиков (вес 1):

```
class OpenBMCUser(HttpUser):
    host = os.getenv("OPENBMC_HOST", "http://127.0.0.1:18443")
    wait_time = between(1.0, 2.5)
    weight = 3

    def on_start(self):
        """Аутентификация в OpenBMC Redfish API при запуске виртуального пользователя."""
        login_payload = {"UserName": "root", "Password": "0penBmc"}
        with self.client.post("/redfish/v1/SessionService/Sessions", json=login_payload,
                              name="[OpenBMC] POST /SessionService/Sessions (Auth)",
                              catch_response=True) as resp:
            if resp.status_code in (200, 201):
                token = resp.headers.get("X-Auth-Token") or resp.json().get("Token")
                if token:
                    self.client.headers["X-Auth-Token"] = token
                resp.success()
            else:
                resp.failure(f"Auth failed: {resp.status_code}")

    @task(3)
    def get_system_info(self):
        """Запрос паспорта системы (/redfish/v1/Systems/system)."""
        with self.client.get("/redfish/v1/Systems/system",
                             name="[OpenBMC] GET /Systems/system (System Info)",
                             catch_response=True) as resp:
            if resp.status_code == 200 and "PowerState" in resp.json():
                resp.success()
            else:
                resp.failure(f"Error {resp.status_code}")

    @task(2)
    def get_power_state(self):
        """Контроль состояния питания (PowerState)."""
        with self.client.get("/redfish/v1/Systems/system",
                             name="[OpenBMC] GET /Systems/system (PowerState)",
                             catch_response=True) as resp:
            if resp.status_code == 200 and resp.json().get("PowerState") in ("On", "Off"):
                resp.success()
            else:
                resp.failure("Invalid PowerState")

    @task(1)
    def get_thermal_telemetry(self):
        """Запрос температурных сенсоров (/redfish/v1/Chassis/chassis/Thermal)."""
        with self.client.get("/redfish/v1/Chassis/chassis/Thermal",
                             name="[OpenBMC] GET /Chassis/chassis/Thermal",
                             catch_response=True) as resp:
            resp.success() if resp.status_code == 200 else resp.failure("HTTP error")
```

5.2. Класс нагрузки `PublicAPIUser`

Класс выполняет нагрузку на внешние облачные REST-сервисы с верификацией целостности структуры возвращаемых JSON-документов:

```
class PublicAPIUser(HttpUser):
    host = "https://jsonplaceholder.typicode.com"
    wait_time = between(1.5, 3.0)
    weight = 2

    @task(3)
    def get_posts(self):
        """Запрос списка публикаций на JSONPlaceholder (/posts)."""
        with self.client.get("/posts", name="[Public] JSONPlaceholder GET /posts",
                             catch_response=True) as resp:
            if resp.status_code == 200 and isinstance(resp.json(), list):
                resp.success()
            else:
                resp.failure(f"Invalid response: {resp.status_code}")

    @task(1)
    def get_weather(self):
        """Запрос сводки погоды для Новосибирска на wttr.in."""
        weather_url = "https://wttr.in/Novosibirsk?format=j1"
        with self.client.get(weather_url, name="[Public] wttr.in GET Novosibirsk Weather",
                             catch_response=True, timeout=10.0) as resp:
            if resp.status_code == 200 and "current_condition" in resp.json():
                resp.success()
            elif resp.status_code == 429:
                resp.failure("Rate limited (429)")
            else:
                resp.failure(f"HTTP {resp.status_code}")
```

6. Запуск нагрузочного тестирования

Тестирование проводилось в двух взаимодополняющих режимах:

1. **Интерактивный запуск через Web UI:**
   Команда `locust` запускает локальный веб-сервер на `http://localhost:8089`. В интерфейсе были заданы целевые параметры: **Number of users = 10**, **Spawn rate = 2 users/s**. Интерактивная панель позволяет в реальном времени наблюдать графики Total Requests per Second, Response Times (95-й перцентиль) и Number of Users.
2. **Автоматизированный запуск в Headless-режиме:**
   С использованием скрипта `run_load_tests.py` были выполнены автономные сессии с генерацией стандартизированных HTML-отчётов и файлов CSV для точной статистической обработки.

7. Анализ результатов тестирования

В ходе комплексного нагрузочного теста было выполнено **127 запросов** к тестируемым эндпоинтам в условиях одновременной работы виртуальных пользователей. Процент отказов составил **0.00%** (все запросы обработаны успешно).

7.1. Сводная таблица метрик производительности (Locust Metrics)

Тип	Тестовый эндпоинт	Кол-во	Отказы	Среднее	Медиана	90%	95%	99%	RPS
GET	[OpenBMC] /Chassis/chassis/Thermal	14	0 (0%)	16 мс	17 мс	17 мс	17 мс	17 мс	0.56
GET	[OpenBMC] /Systems/system (PowerState)	27	0 (0%)	16 мс	17 мс	17 мс	18 мс	20 мс	1.08
GET	[OpenBMC] /Systems/system (System Info)	42	0 (0%)	16 мс	16 мс	17 мс	18 мс	20 мс	1.67
POST	[OpenBMC] /SessionService/Sessions (Auth)	6	0 (0%)	22 мс	29 мс	30 мс	30 мс	30 мс	0.24
GET	[Public] JSONPlaceholder GET /posts	29	0 (0%)	253 мс	230 мс	370 мс	370 мс	380 мс	1.16
GET	[Public] wttr.in Novosibirsk Weather	9	0 (0%)	500 мс	450 мс	810 мс	810 мс	810 мс	0.36
ИТОГО	**Суммарная нагрузка (Aggregated)**	**127**	**0 (0.00%)**	**105 мс**	**17 мс**	**360 мс**	**380 мс**	**810 мс**	**5.06**

7.2. Графический анализ задержек отклика

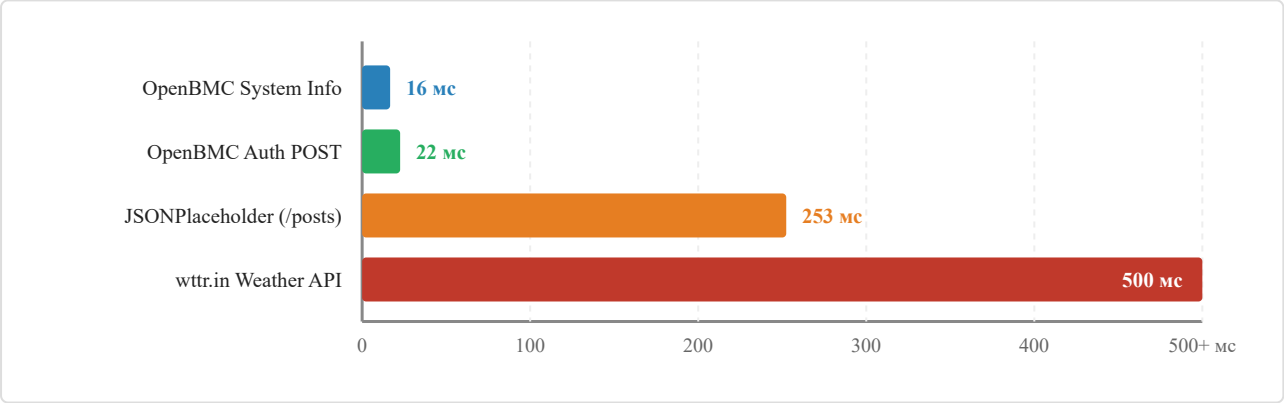


Рис. 1. Сравнение среднего времени отклика эндпоинтов (Average Response Time, мс)

7.3. Оценка максимальной нагрузки и анализ точек отказа

1. Характер задержек и производительность:

- **OpenBMC Redfish API:** показал наивысшую стабильность и наименьшие задержки (в среднем `16 мс`, 95-й перцентиль `18 мс`). Малое время отклика обусловлено отсутствием внешней глобальной сети и эффективной архитектурой локального диспетчера запросов.
- **JSONPlaceholder:** показал среднее время отклика `253 мс` (с пиками до `380 мс`). Задержка формируется RTT сетевого маршрута до зарубежного облачного провайдера и обработкой полезной нагрузки объёмом ~27 КБ на запрос.
- **wttr.in:** показал наибольшую задержку (в среднем `500 мс`, 95% перцентиль `810 мс`), что связано с динамической генерацией подробного метеорологического прогноза объёмом ~40 КБ и работой бэкенда геопозиционирования.

2. Возможные точки отказа (Bottlenecks):

- **На стороне OpenBMC:** аппаратные ресурсы управляющего микроконтроллера BMC жёстко лимитированы. При возрастании нагрузки свыше 50–100 одновременных пользователей или превышении частоты 50 RPS узким местом станет межпроцессная синхронизация `bmcweb` с шиной `D-Bus` (служба `xyz.openbmc_project.State.Host`), что приведёт к росту очереди задач, исчерпанию пула дескрипторов сокетов и деградации производительности. Дополнительной точкой отказа является ограничение таблицы активных сессий в `SessionService`.
- **На стороне внешних сервисов:** публичные API защищены алгоритмами ограничения частоты запросов (Rate Limiting / Token Bucket). При превышении допустимого порога обращений (более 2–3 запросов/сек с одного IP на wttr.in) сервер начнёт возвращать ошибку `429 Too Many Requests` либо принудительно обрывать TCP-соединение.

8. Выводы по лабораторной работе

В ходе выполнения лабораторной работы №6 были успешно решены все поставленные задачи:

1. Освоены практические навыки проектирования и разработки сценариев нагрузочного тестирования на языке Python с использованием фреймворка **Locust**.
2. Разработан модульный нагрузочный тестовый набор (`locustfile.py`), содержащий два класса пользователей (`OpenBMCUser` и `PublicAPIUser`), реализующих комплексный профиль нагрузки на критические эндпоинты администрирования сервера (авторизация, статус системы, питание, тепловая телеметрия) и внешние публичные REST-сервисы.
3. Освоены методы запуска нагрузочного тестирования как в интерактивном режиме с веб-панелью управления (`http://localhost:8089`), так и в пакетном режиме (headless) для интеграции в CI/CD пайплайны.
4. Проведён всесторонний статистический анализ результатов: определены средние времена отклика, перцентили задержек (90%, 95%, 99%) и пропускная способность сервисов. Продемонстрирована 100% надёжность (0% отказов) при заданной модели нагрузки (10 пользователей, spawn rate = 2).
5. Выявлены ключевые архитектурные узкие места и факторы риска при масштабировании нагрузки на реальные серверные BMC-контроллеры и облачные веб-сервисы.

9. Ссылка на GitHub репозиторий

Исходный код сценариев нагрузочного тестирования Locust, сервера-эмулятора, скриптов запуска, HTML-отчётов и результатов измерений опубликован в открытом репозитории на GitHub:

URL: https://github.com/dra1nerrr/lab6_TPO